

ERS-ERE System Technical Contract

Message-Based Resolution Interfaces, Idempotent Processing, and Canonical Identifier Semantics

Status:

Stable

Version:

0.2

Date: 23 Feb 2026

Document status, versioning, change log

Version	Date	Change description	Author
0.1.0-rc.1	23/12/2025	Initial ERS-ERE contract baseline	Meaningfy
0.1.0-rc.2	16/01/2026	Addressed live comments.	Meaningfy
0.2.0-rc.1	06/02/2026	Added sections on Canonical Identifier derivation, and interaction model independency. Addressed comments and decisions from a live alignment sessions.	Meaningfy
0.2.0-rc.2	23/02/2026 (started)	Document aligned with recent decisions: ERE designated as cluster membership authority; canonical ID and cluster ID treated as equivalent; proposed placements updated; notification responses revised; full rebuild removed as a request type; similarity score added.	Meaningfy

Table of contents

Introduction	4
Canonical Identifier Derivation and Stability (Normative)	4
Contract scope and realisation model	5
Interaction model and idempotency	6
The ERE Interface	8
Requests channel.....	8
Responses channel.....	8
Entity Mention.....	8
Entity Mention Identifiers	9
Relationship to Canonical Identifiers.....	9
Entity Mention Resolution Request.....	10
Entity Mention Resolution Response.....	10
Resolution Request with proposed placements	11
ERE timeouts and singleton proposals.....	12
Resolution Request with Rejections	12
Resolution Request with (possibly multiple) Proposals and Rejections	13
Update Responses.....	13
(Non-Normative) Full cluster rebuild	13
(Normative) Communication of rebuild outcomes	13
Error Response	14
The ERE Data Model	14
LinkML Code	15
Functional requirement coverage matrix	15
Examples and Tests	17
Configuration.....	18

Introduction

The Entity Resolution Engine (ERE) is a service that provides entity resolution capabilities. Together with the Entity Resolution Service (ERS), it forms the Entity Resolution System (ERSys), whose purpose is to enable human users and external systems to resolve entities within their data in a consistent and interoperable manner.

Within this architecture, the ERE is responsible for *clustering logic and canonical identity materialisation*, while the ERS is responsible for *controlled exposure, time-budget discipline, and persistence of the latest recorded outcomes*. Their cooperation is governed exclusively through the technical contract defined in this document.

The ERE accepts structured entity mentions as input, including descriptions of organisations, persons, publications, or other domain entities. These descriptions are provided as text-based serialisations, for example RDF/XML, RDF/Turtle, or XML, with the concrete format indicated through a MIME type. Based on this input, the ERE performs clustering according to its internal resolution logic and produces clustering outcomes consisting of a cluster identifier and associated assessment signals such as similarity and confidence scores. The cluster identifier generated by the ERE constitutes the *canonical identifier* exposed by ERS.

Although initially developed within the TED-SWS context, the ERE is designed as an independent and replaceable component. Any implementation that conforms to the normative contract defined herein may interoperate with ERS. This contract establishes a stable and implementation-independent boundary between ERS and ERE. It defines the message structures, correlation semantics, idempotent processing expectations, and canonical identifier derivation rules, without prescribing internal algorithms, data structures, or optimisation strategies.

Multiple ERE implementations may exist, ranging from baseline to advanced variants. All such implementations shall conform to this contract. By stabilising the boundary semantics, ERSys preserves *engine replaceability*, architectural clarity, and long-term evolvability.

Canonical Identifier Derivation and Stability (Normative)

Stable and reproducible identifier semantics are essential for asynchronous processing, idempotent request handling, and consistent exposure of clustering outcomes. This section defines the normative rules governing canonical identifier value derivation and stability within ERSys.

Within ERSys, a single identifier value space is used. The *cluster identifier generated by the ERE constitutes the canonical identifier* exposed by ERS. There is no separate or elevated identifier layer. The same identifier value is used consistently for clustering, exposure, replay, and downstream reference.

Canonical identifier values shall be deterministically derived from the components of the entity mention identifier, namely: `(source_id, request_id, entity_type)`.

The normative derivation rule is:

```
SHA256(concat(source_id, request_id, entity_type))
```

All conformant ERS and ERE implementations shall implement this derivation rule identically. No component shall introduce alternative derivation strategies, centralised allocation mechanisms, or non-deterministic identifier generation.

Deterministic derivation ensures that:

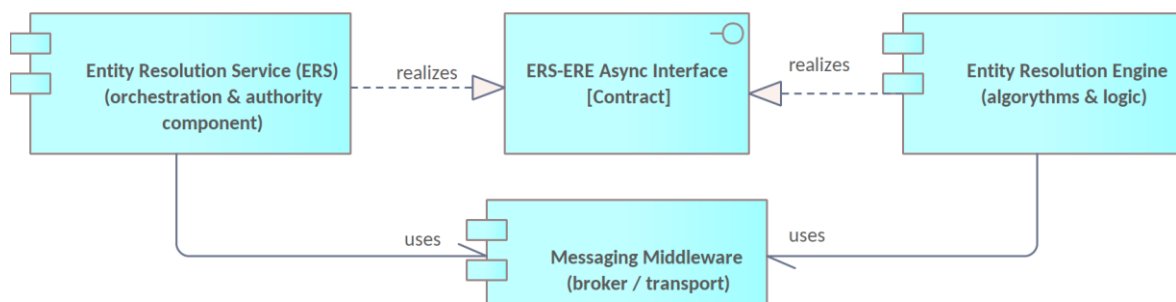
- repeated or replayed resolution requests produce stable identifier values;
- provisional singleton clusters generated under time-budget constraints remain stable if later confirmed;
- clustering outcomes can be correlated across asynchronous exchanges without additional coordination.

Identifier value stability is distinct from cluster membership stability. While the identifier value of a cluster remains stable once generated, the set of entity mentions assigned to that cluster may evolve over time as new evidence is processed or re-resolution occurs. Conversely, an entity mention may be reassigned to a different cluster as clustering outcomes change. Stability therefore applies to identifier values, not to membership immutability.

The purpose of this approach is to eliminate centralised identifier governance and prevent tight coupling between ERS and ERE. By relying on a shared deterministic derivation rule, ERSys enables independent evolution of ERE implementations while preserving interoperability, idempotency, and long-term referential stability.

This section is the sole normative specification of canonical identifier value derivation within ERSys. All other sections and artefacts shall reference this rule rather than restating or redefining it.

Contract scope and realisation model



This diagram illustrates the structural scope of the ERS-ERE technical contract and clarifies the responsibilities of the participating components.

The ERS–ERE contract defines an asynchronous application interface governing the exchange of resolution requests and clustering outcomes between ERS and ERE. It constitutes the sole normative agreement between the two components and defines their cooperation independently of any specific implementation.

Both ERS and ERE realise this contract, but with distinct responsibilities. The ERE holds *clustering authority*: it evaluates entity mentions, executes its internal resolution logic, and produces clustering outcomes, including the cluster identifier that constitutes the canonical identifier. The ERS holds *exposure and integration authority*: it forwards resolution and re-resolution requests, enforces client-facing time budgets, records the latest clustering outcome per mention, and exposes canonical identifiers through its public interfaces.

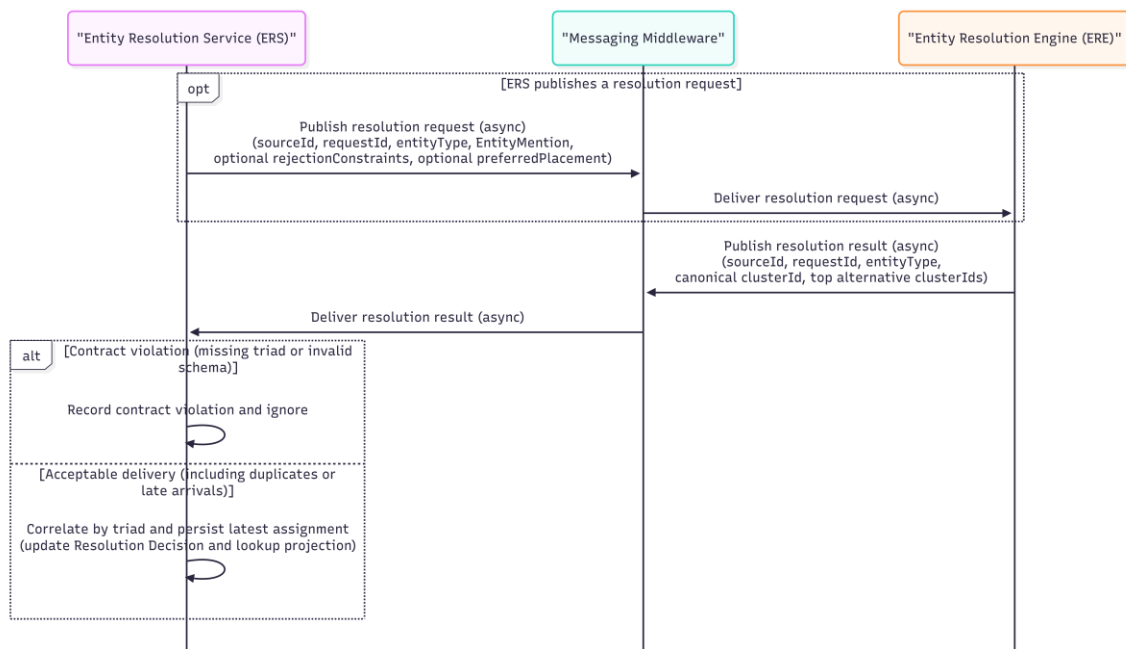
The contract is intentionally decoupled from the messaging middleware used for transport. While ERS and ERE exchange messages via a broker or messaging infrastructure, the choice of technology, delivery guarantees, and deployment topology are outside the scope of this contract. The contract specifies what messages are exchanged and under which semantic rules, not how they are transported.

This separation ensures that the contract remains stable even if the messaging infrastructure evolves, that ERS and ERE implementations remain independently replaceable, and that clustering authority, exposure responsibility, and transport concerns are kept clearly distinct.

The detailed interaction semantics governed by this contract are described in the following sections.

Interaction model and idempotency

The ERE exposes its functionality through an asynchronous publish/subscribe interaction model. The ERS publishes resolution and re-resolution requests to a requests channel, and the ERE subscribes to this channel to receive and process them. The ERE, in turn, publishes clustering outcomes to a responses channel, which the ERS subscribes to in order to receive and integrate the results.



The primary interaction governed by this contract is the entity resolution request. In response, the ERE produces a *clustering outcome* consisting of a primary cluster assignment for the entity mention and associated assessment signals, such as similarity and confidence scores. Where applicable, alternative cluster identifiers may also be included to support transparency and downstream reasoning.

This interaction model is intentionally asynchronous. Submitting a request does not yield an immediate response; instead, clustering outcomes are published at a later time once computation completes. This reflects the computational variability of entity resolution and allows ERS to enforce client-facing time budgets without blocking on engine execution.

Because communication is asynchronous and delivery guarantees may be at-least-once, identical requests or responses may be observed multiple times. All conformant implementations shall therefore implement *idempotent processing semantics*. Repeated submissions of the same request shall not create duplicate state, and repeated delivery of the same clustering outcome shall not result in inconsistent assignments.

Clustering authority resides in the ERE. The cluster identifier produced in a clustering outcome constitutes the canonical identifier. Upon receipt of a valid outcome, the ERS correlates it using the entity mention identifier triad and records the *latest received clustering outcome* for that mention. Integration discipline is applied at the ERS boundary, but canonical identity is determined by the ERE.

Outcomes may arrive without a preceding request in cases of engine-initiated reclustering. Such outcomes are processed under the same correlation and idempotency rules.

The interaction model defined here specifies the semantic exchange between ERS and ERE. It does not mandate a particular messaging middleware or transport technology. Any infrastructure may be used, provided that message ordering, duplication, and delivery characteristics are handled in a manner consistent with the idempotent semantics defined in this contract.

[Informative] A candidate system that we are considering for the implementation of this model is [Redis queues](#).

The ERE Interface

In this section, we list the ERE channels mentioned above and the requests they support. The data types that the requests exchange are briefly mentioned in this section and formally described in a separate LinkML schema.

Requests channel

Name: `ere_requests`

As mentioned above, this receives ERE requests and internally enqueues them for asynchronous execution.

Namely, the channel is used to send instances of the `ERERequest` subclasses in JSON format (see the next section for a description of the interface data model).

Responses channel

Name: `ere_responses`

This is the counterpart of the requests channel. Results are published/pushed asynchronously on this channel when the ERE completes the execution of a request.

The channel is used to publish instances of the `EREResponse` subclasses in JSON format.

Note that this includes the management of errors (see below).

Entity Mention

An entity mention represents data submitted to the system by a source system or actor that describes, or refers to, a real-world entity. An entity mention is always conveyed as part of an entity resolution request and constitutes the subject of the resolution operation.

An entity mention consists of:

- an **entity mention identifier**, used for correlation and scoping of the request, and
- an **entity representation**, provided as a text-based serialisation together with its declared MIME type.

This is an example:

```
{
  "identifier": {
    "request_id": "324fs3r345vxab",
    "source_id": "TEDSWS",
    "entity_type": "http://www.w3.org/ns/org#Organization",
  },
  "content": "epd:ent005 a org:Organization; ... cccev:telephone \"+44 1924306780\" .",
}
```



```
"content_type": "text/turtle"  
}
```

Entity Mention Identifiers

Each entity mention shall be associated with an entity mention identifier. The identifier is a structured value composed of the following triple:

- `source_id` – identifies the originating system or actor,
- `request_id` – uniquely identifies the request within the scope of the originating system,
- `entity_type` – identifies the type of entity being described.

These values are provided by the ERS client and **shall be present** in every entity resolution request. Together, the triple uniquely identifies an entity mention within the scope of the ER System.

Entity mention identifiers serve the following purposes:

- correlation of asynchronous requests and responses (the ERE specifies with an additional `ere_request_id` field, to deal with multiple requests or responses about the same entity, see below),
- enforcement of idempotent request handling,
- traceability of resolution proposals and decisions across system components.

An entity mention identifier shall not be interpreted as a canonical identifier and shall not carry governance semantics. Its role is limited to request scoping and correlation.

Relationship to Canonical Identifiers

Canonical identifiers are not defined or governed within the entity mention identifier itself. The rules governing canonical identifier value derivation and stability are specified in the normative section *Canonical identifier derivation and stability*.

The components of the entity mention identifier (`source_id`, `request_id`, `entity_type`) are used as inputs to the standardised deterministic derivation rule that defines the canonical identifier value space. Deterministic derivation ensures that identifier values are reproducible across ERS and ERE implementations and across repeated or replayed requests.

The canonical identifier for an entity mention is established through the clustering outcome produced by the ERE. The cluster identifier generated by the ERE constitutes the canonical identifier exposed by ERS. Identifier generation according to the deterministic rule does not by itself imply cluster membership; canonical status arises only when the ERE assigns an entity mention to a cluster in a valid clustering outcome.

By relying on a shared deterministic value space, ERSys ensures that identifier values remain stable across asynchronous processing, idempotent handling, provisional issuance under time-budget constraints, and subsequent clustering updates.

For the normative specification of canonical identifier value derivation and stability guarantees, refer to the section *Canonical identifier derivation and stability*.

Entity Mention Resolution Request

This is the main service the ERE provides. An entity resolution request is sent to the requests channel, and it contains details of an entity mention for which the ERE finds equivalence cluster candidates. The ERE replies with a resolution response object (an instance of `EntityMentionResolutionResponse`) sent to the responses channel.

The following is an example from the LinkML code (see '[The ERE Data Model](#)' section). More details about it are explained below.

```
{
  "type": "EntityMentionResolutionRequest",
  "entity_mention": {
    "identifier": {
      "request_id": "324fs3r345vx",
      "source_id": "TEDSWS",
      "entity_type": "http://www.w3.org/ns/org#Organization"
    },
    "content": "epd:ent005 a org:Organization; ... cccev:telephone \"+44 1924306780\" .",
    "content_type": "text/turtle"
  },
  "timestamp": "2026-01-14T12:34:56Z",
  // As said, we need this internal ID and it can be auto-generated (eg, with UUIDs)
  "ere_request_id": "324fs3r345vx:01"
}
```

As you can see, the request contains an entity mention, which in turn, has a composite identifier.

Note on `ere_request_id`: this is an ERE-specific field, which the ERE needs to match responses to requests. In fact, we might have two requests containing the same entity mention and hence, this disambiguation field is needed too.

For the purposes of idempotent handling, a resolution request is correlated by its entity mention identifier together with any applicable recommended placements (such as rejected canonical identifiers).

Entity Mention Resolution Response

This is the counterpart of the resolution request, which the ERE sends to the entity resolution channel asynchronously. This object contains a set of references to candidate clusters, together with confidence scores. The meaning of them is that the input entity is considered equivalent to the referred clusters, with the given score.

```
{
  "type": "EntityMentionResolutionResponse",
  "entity_mention_id": {
    "request_id": "324fs3r345vx",
    "source_id": "TEDSWS",
    "entity_type": "http://www.w3.org/ns/org#Organization"
  },
  "candidates": [
    {
```

```

    "clusterId": "324fs3r345vx-aa32wa",
    "confidenceScore": 0.91,
    "similarityScore": 0.85
  },
  {
    "clusterId": "324fs3r345vx-bb45we",
    "confidenceScore": 0.65,
    "similarityScore": 0.8
  }
],
"timestamp": "2026-01-14T12:34:59Z",
"ere_request_id": "324fs3r345vx:01"
}

```

[Informative] one way to use multiple canonical identifier candidates is to feed the curation application, so that the user can decide which entity is actually equivalent to the input, or they can reject all the candidates (see below).

As you can see, the response reports the composite entity ID of the mention it refers to. This isn't strictly needed, since it could be reconstructed by going back to the original request through `ere_request_id`. Nonetheless, we repeat these data in the response for convenience.

Note that the ERE contract does not concern with returning any entity representations associated to the clusters, e.g., if an organisation O1 might be associated to a cluster CL1, which of canonical entity is CO1, then the ERE will return a cluster ID for CL1, without any representation about the entity CO1 (and, as described above, the cluster ID will be the same as the canonical ID of CO1).

The ERE is the authority to determine the membership of an entity to clusters.

It is the ERS responsibility to manage and govern the canonical entities, including IDs, integration of entity mention memberships, and if necessary canonical representations.

This follows from the normative derivation rule defined in the section *Canonical Identifier Derivation and Stability*.

Resolution Request with proposed placements

A resolution request might contain a list of proposed cluster IDs. The ERE might take them into account, but that is not mandatory, the ERE can still decide a different entity classification and it remains the ultimate authority to decide the clustering of an entity.

This is an example of such request:

```

{
  "type": "EntityMentionResolutionRequest",
  "entity_mention": {
    "identifier": {
      "request_id": "324fs3r345vx",
      "source_id": "TEDSWS",
      "entity_type": "http://www.w3.org/ns/org#Organization"
    },
    "content": "epd:ent005 a org:Organization; ... cccev:telephone \"+44 1924306780\" .",
    "content_type": "text/turtle"
  },
  "proposed_cluster_ids": [
    "324fs3r345vx-aa32wa", "324fs3r345vx-bb45we"
  ]
}

```

```

"timestamp": "2026-01-14T12:34:56Z",
// As said, we need this internal ID and it can be auto-generated (eg, with UUIDs)
"ere_request_id": "324fs3r345vx:01"
}

```

Whether the order of elements in `proposed_cluster_ids` is relevant or not depends on the ERE implementation (i.e., it might decide to give more importance to the top elements).

ERE timeouts and singleton proposals

An ERE client might place just one element in `proposed_cluster_ids` and that might be `canonicalID(entityMention.identifier)`. In other words, it can be proposed that an entity mention is binned into a singleton cluster, containing the entity only.

(Non-normative): we expect the ERS to do this when the ERE doesn't reply to an initial resolution request within a configured timeout. In fact, in that case, the ERS might decide to continue with the initial entity as the possible canonical one. As above, the ERE doesn't have any obligation to classify the entity in this proposed way. Furthermore, given the asynchronous nature of the ERE, an answer to the initial request might arrive after the timeout and the client might decide to consider it despite the subsequent singleton request.

Resolution Request with Rejections

An entity resolution request can include the parameter `excludedClusterIds`, to indicate that a previous proposed clustering is not deemed valid and alternative resolutions are requested. One use case for this operation is to account for a negative curator's decision on an automatic alignment.

This is an example of request with rejections:

```

{
  "type": "EntityMentionResolutionRequest",
  "entity_mention": {
    "identifier": {
      "request_id": "324fs3r345vxab",
      "source_id": "TEDSWS",
      "entity_type": "http://www.w3.org/ns/org#Organization",
    },
    "content": "epd:ent005 a org:Organization; ... cccev:telephone \"+44 1924306780\" .",
    "content_type": "text/turtle"
  },
  "excluded_cluster_ids": [
    "324fs3r345vx-bb45we",
    "324fs3r345vx-cc67ui"
  ],
  "timestamp": "2026-01-14T12:40:56Z",
  "ere_request_id": "324fs3r345vxab:01"
}

```

As for the proposed placements, rejections are suggestions for the ERE, depending on the implementation, this may or may not consider them, and it may decide to store them to gather heuristic and use it in machine learning procedures.

Resolution Request with (possibly multiple) Proposals and Rejections

A request may have both proposed placements and rejections. Both may be used to improve the clustering, by combining the behaviours described above.

Similarly, the ERE may accumulate subsequent requests with proposed placements or rejections, by considering the merge of all the proposals coming from multiple messages.

Update Responses

The ERE may change cluster assignment of entities based on its internal maintenance, periodic reviews of previously computed data, full rebuilds (see below). When this happens, the ERE notifies all subscribing clients by pushing a `EntityMentionResolutionResponse` message on the response channel. Since there is no request that can be related it, this response has an `ere_request_id` value having a special prefix ``ereNotification:'`.

The alternative could be having a completely different type of message that is not a subclass of `EREResponse` (see the data model below). However, using the convention above allows for using the same response type that is used to react to client requests and keep the ERE contract simpler.

(Non-Normative) Full cluster rebuild

The ERE may support internal reprocessing or rebuild operations that trigger re-evaluation of previously computed clustering outcomes. Such operations are entirely internal to the ERE and are not invoked through the asynchronous request channel defined in this contract.

A rebuild may be scoped to one or more entity types. If no scope is specified, the operation may apply to all entity types managed by the ERE. The exact internal mechanics of rebuild depend on the specific ERE implementation and its resolution strategies. This contract does not prescribe how reprocessing is performed, how intermediate state is managed, or how clustering algorithms are re-executed.

Reprocessing may be initiated manually by an operator or automatically by the ERE as part of maintenance, optimisation, or algorithm updates.

(Normative) Communication of rebuild outcomes

Although rebuild is not triggered via the asynchronous request channel, any change in clustering outcomes resulting from rebuild shall be communicated through the standard responses channel using the same message structures defined for resolution outcomes.

Rebuild-triggered clustering updates shall follow the same correlation and idempotency rules as regular responses. In particular:

- Updates shall reference the entity mention identifier triad.

- Duplicate or repeated updates shall be processed idempotently.
- The ERS shall record the latest received clustering outcome per entity mention.

This contract does not define rollback semantics, temporal state restoration, or lineage relationships between clusters. The ERE is not required to preserve historical cluster evolution or to expose previous cluster states.

Rebuild operations may result in clustering changes being published concurrently with new resolution requests. Implementations shall ensure that such interleaving does not violate idempotent processing guarantees or correlation rules.

The purpose of this section is to clarify that reprocessing is an ERE responsibility, that its effects are propagated through standard outcome messages, and that no additional contract surface is introduced for rebuild control or state rollback.

Error Response

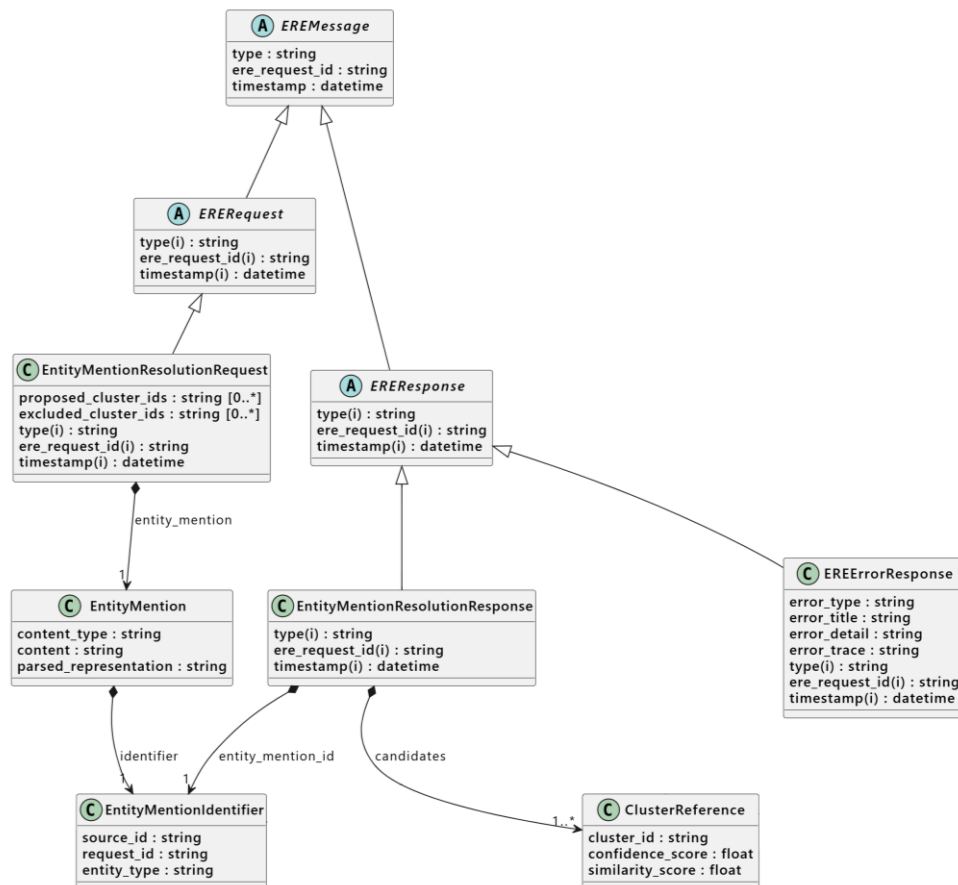
An ERE request of any type (resolution, rebuild) might fail, due to problems like a malformed request or an internal malfunction. In this case, the ERE will publish a response object of type `ERRErrorResponse`. This is an example:

```
{
  "type": "ERRErrorResponse",
  "ere_request_id": "324fs3r345vx",
  "errorType": "ere.exceptions.MalformedRequestError",
  "errorTitle": "The entity data is missing in the request",
  "errorDetail": "The 'entity' attribute is required in EntityResolutionRequest message",
  // Optional and not recommended for production use
  "errorTrace": "Traceback (most recent call last):\n  File \"/app/ere/service.py",
    line 45, in process_request\n..."
}
```

The ERE Data Model

The data types used for the operations described in the previous model are formally defined using the [LinkML schema language](#).

This is a PlantUML illustration of the model classes (automatically generated from the LinkML code, see links below):



LinkML Code

The LinkML source files is [on github](#). From this, we auto-generate:

- [Navigable docs](#)
- [JSON schema](#), which we plan to use in tasks like defining REST APIs (i.e., import in OpenAPI schemas)
- [Python/Pydantic definitions](#), which we plan to use to define the ERE service interfaces and implementations.

Functional requirement coverage matrix

The following table provides a bigger picture by showing how the designed contract between the ERS and the ERE satisfies the functional requirements specified for the entire system in the Product Requirements Document. It allows the tracing of requirement fulfilment and shows how they are handled in the ERS–ERE communication context.

Functional requirement	How it affects the ERS-ERE contract (a.k.a. required operations)	How it's handled by ERE/the contract
------------------------	--	--------------------------------------

Entity resolution (single and bulk resolution)	- Entity resolution request is submitted - Possible outcome: - Entity resolution response is passed to ERS - Entity resolution error occurs - The ERE doesn't respond within a timeout (see below)	- An EntityMentionResolutionRequest is published to the ere_requests channel. - Outcomes: - The ERE publishes an EntityMentionResolutionResponse to ere_responses - The ERE publishes an EREErrorResponse to the ere_responses channel
Entity resolution with proposed placement	- An entity resolution was submitted by the ERS - The ERE didn't reply in time (according to a configured timeout) - The ERS re-submit a new request, proposing canonicalID(entity) as singleton cluster placement	- The ERE might decide to accept the singleton creation or not. - The response to the former request might eventually arrive after the timeout and the ERS might decide to use or ignore it. - This might or might not be a singleton
Bulk refresh	Handled by ERS, ERE is not affected.	N.A.
Retrieval of entity data (for Link Curation app)	ERE is not affected, data provided by ERS from its store.	N.A.
Re-resolve a selected entity mention (refresh)	<i>See Manual curation: Rejection of proposed and alternative alignments (Link Curation app).</i>	
Recalculate the whole	The operation is internal to the ERE, doesn't affect the contract.	The exact meaning of the operation depends on the ERE implementation.

ERE cluster structure		
Manual curation: Acceptance / rejection of proposed alignments (Link Curation app)	Manual curation decisions are recorded by the ERS and passed to the ERE as suggestions.	The propagation of the user feedback to the ERE is done through the usual resolution flow, using either <code>proposed_clusters_ids</code> or <code>excluded_cluster_ids</code> (see the <i>Entity resolution with proposed placement</i> row).
Manual curation: Acceptance of alternative alignment (Link Curation app)	The ERE is not affected. Same as default acceptance.	N.A.
Manual curation: presentation of stats of resolution	The ERE is not affected by this.	N.A.
Manual curation: User management	The ERE is not affected by this.	N.A.

Examples and Tests

Examples of ERE message exchanges [are provided in the ERS GitHub repository](#). These are mock-up cases based on [collected real data](#).

A summary of the available examples:

- Example 1: Simple resolution case -- Organisations with Minor detail variations
 - Same legal name, address, contact details (Bulgarian Commission on Protection of Competition)
- Example 2: Negative example or resolution -- Different procedure numbers
 - ✗ Similar descriptions (both nursing services), but different procedure numbers in the titles
 - ✗ The ERE creates new cluster Identifier (rejected candidate at 0.68)
- Example 3: Resolution with exclusions -- Organisations with Minor detail variations
 - ✗ Submission of resolution request with two URIs previously provided by ERE marked as excluded
 - ✗ The ERE creates new canonical Identifier

Additionally, [Gherkin tests](#) are available [on github](#) to clarify the behaviour explained in this document.

Configuration

The ERE is configured via YAML, possibly with mechanisms to [inject environment variables](#) in YAML files. Here, we list some of the configuration options that any ERE needs to support to tune the interface described in the previous sections. Further configuration might be needed depending on the specific ERE implementation (e.g. Redis connection coordinates).

entity_types: URI list of CETs, ie, the entity types that the ERE supports for resolution. In the initial basic ERE, these will likely be organisations and procedures.

resolution_timeout (integer, seconds): max time the ERE should spend trying to resolve an entity, before being interrupted with a failure. Optionally, multiple time duration formats can be supported for this, eg, accepting values like “1m 5s”.

confidence_threshold (float, range 0-1): Minimal value of similarity score needed for an entity mention to be associated with a canonical entity through a canonical identifier. We however recommend not to use such parameter or keep it near to zero to prevent returning empty resolution responses. The long tail problem is handled by the ERS pruning to top-N options.